Complete Website Documentation

1. PROJECT OVERVIEW

Project Name: Modena E-Commerce Platform
Purpose: A headless e-commerce storefront for premium kitchenware and home appliances.
Business Objective: Provide a seamless, modern, "Apple-like" shopping experience to increase conversions and customer engagement.
Target Users: Customers purchasing premium kitchenware and heavy-duty home appliances in India.
Main Features:

- Headless architecture with fluid responsive design.
- Intelligent RAG AI Chatbot Assistant (Alex).
- Dynamic product catalog powered by WooCommerce.
- Multi-step OTP and JWT authentication.
- Seamless single-page cart and Razorpay checkout.
- Account management (Orders, Returns, Replacements).
  Overall Architecture: A decoupled architecture with a React/Vite SPA on the frontend, communicating via REST APIs to a Headless WordPress/WooCommerce database, and a FastAPI Python backend handling the RAG AI chatbot pipeline.

2. TECHNOLOGY STACK

Frontend (Client-Side)

- **React 19:** UI framework for building the SPA.
- **Vite:** Build tool and development server.
- **Tailwind CSS v4:** Utility-first CSS framework for responsive styling.
- **Lucide React:** SVG icon library.
- **SWR:** React hooks for data fetching and caching.
- **jsPDF & html2canvas:** Generating invoice PDFs.

Backend & API

- **FastAPI (Python):** Serves the AI Chatbot backend (`rag-backend`).
- **Uvicorn:** ASGI web server implementation for FastAPI.
- **LangChain:** Framework for developing applications powered by language models.
- **ChromaDB:** Vector database for semantic search and Retrieval-Augmented Generation (RAG).
- **Sentence-Transformers:** Generating text embeddings (`all-MiniLM-L6-v2`).
- **PyJWT:** JSON Web Token implementation in Python for secure API sessions.

CMS & Commerce Engine

- **WordPress:** Headless CMS.
- **WooCommerce:** E-commerce engine handling products, categories, orders, and reviews.
- **MySQL/MariaDB:** Database via LocalWP with High-Performance Order Storage (HPOS) enabled.

Authentication & Third-Party

- **JWT Auth (WordPress plugin):** Issues tokens for REST API authentication.
- **Razorpay:** Payment gateway integration for handling transactions.

3. WEBSITE STRUCTURE

Pages and Routes

The application is structured as a Single Page Application (SPA). Views are managed via state (`currentView`) rather than strict URL routing, maintaining context seamlessly.

- **Homepage (`home`)**:
  - **Purpose:** Main landing page.
  - **Components:** Hero Banner, Categories, Signature Collection, Product Sliders, Philosophy/About, Footer.
- **Products / Store (`products`)**:
  - **Purpose:** Browse the full catalog.
  - **Data:** Fetches product lists and categories dynamically.
- **Product Details (`productDetails` / `selectedProduct`)**:
  - **Purpose:** Detailed view of a single product.
  - **Components:** Image gallery, Price, Description, Reviews (average rating), Add to Cart.
- **Your Orders (`yourOrders`)**:
  - **Purpose:** Customer dashboard for order management.
  - **Actions:** View history, request returns, request replacements.
- **Authentication Modals (Login/Register)**:
  - **Purpose:** User sign-up and login via email/OTP and passwords.
- **Cart & Checkout Drawers/Modals**:
  - **Purpose:** Review selected items and process payment via Razorpay.

4. HOMEPAGE

The homepage is constructed dynamically using modular sections:

- **Hero Banner:** Displays top-tier products fetched from the `hero-banner` WooCommerce category.
- **Product Sections (Signature Collection, Electronics, Utensils):** Dynamically filters products based on WooCommerce category slugs (`electronics`, `utensils`, etc.).
- **CTA Sections:** Promotional blocks to drive conversions.
- **Philosophy / About:** Static section communicating brand values.
- **Footer:** Links to policies, social media, and navigation.
- **Floating Cart Button:** Dynamically positions itself based on scroll depth to prevent overlapping the footer.
- **Chatbot (Alex):** Floating AI assistant widget available on all pages.

5. HERO BANNER SYSTEM

- **Source:** WordPress "Hero Banner" category (slug: `hero-banner`).
- **Product Selection:** Any product tagged with this category is dynamically loaded into the Hero slider.
- **Data Extracted:** Product Title (decoded HTML entities), Product Image, Short Description, and Price.
- **Image Behavior:** On desktop, the image sits on the right with a seamless gradient mask. On mobile, the image sits behind a dark gradient backdrop to ensure text readability while maintaining product visibility.
- **Text Handling:** HTML entities (like `&#215;`) are programmatically decoded for clean display.

6. PRODUCT SYSTEM

- **Database:** WooCommerce REST API (`/wp-json/wc/store/v1/products`).
- **Product Cards:** Display image, title, price, and category. Includes a hover-state "Add to Cart" or "View Details" button.
- **Categories:** Extracted from the WooCommerce category taxonomy.
- **Prices:** Parsed from `price_html` or `price` fields.
- **Descriptions:** Stripped of HTML tags and truncated via CSS line-clamp.
- **Filtering:** Users can search and filter by category tags.
- **Out of Stock:** Grayed out cards to signify unavailability (detailed below).

7. OUT-OF-STOCK SYSTEM

- **Implementation:** Products assigned to the "Out of Stock" category (slug: `out-of-stock`) in WooCommerce.
- **Frontend Behavior:**
    - The product card is visually grayed out and desaturated.
    - An "OUT OF STOCK" badge is overlaid.
    - The "Add to Cart" button is disabled or hidden to prevent purchases.
- **Data Flow:** The `useProducts` hook identifies the category string and flags the `isOutOfStockCategory` boolean.

8. REVIEWS & RATINGS

- **System:** Native WooCommerce Reviews API (`/wp-json/wc/v3/products/reviews`).
- **Calculation:** Average rating and review count are pulled directly from the product object (`average_rating`, `rating_count`).
- **No Reviews Behavior:** If `rating_count` is 0, the UI explicitly displays "0.0 (0 Reviews)" or "No reviews". It does not generate fake data.
- **Display:** Star ratings are rendered dynamically based on the average score.
- **Submission:** Verified users can submit a review via a POST request to the WooCommerce reviews endpoint.

9. CART & CHECKOUT

- **State:** Managed locally in React state (`cart` array) with persistence to `localStorage`.
- **Interactions:** Users can increase/decrease quantities or remove items.
- **Pricing:** Calculates subtotal dynamically based on the parsed numeric prices of cart items.
- **Checkout Process:**
    1. User fills out shipping/billing details.
    2. Frontend requests a Razorpay order ID (`/wp-json/modena/v1/create-razorpay-order`).
    3. Razorpay payment modal opens.
    4. On success, payment signature is verified (`/wp-json/modena/v1/verify-razorpay-payment`).
    5. WooCommerce Order is created via API.

10. ORDERS, RETURNS & REPLACEMENTS

- **Navigation:** Accessible via the "Orders" and "Returns" buttons located strategically below the main header on desktop, and via the mobile menu.
- **Order History:** Displays past orders pulled from WooCommerce.
- **Returns:** Users can initiate a return by providing a reason and uploading proof (images) via `/wp-json/modena/v1/upload-return-proof`.
- **Replacements:** Similar flow to returns, targeting a replacement action rather than a refund.

11. CHATBOT / AI SYSTEM

- **UI:** A floating chat window toggled via a bottom-right icon.
- **Backend:** FastAPI RAG Server running locally.
- **AI Model:** Uses Google's Gemini LLM via LangChain, grounded in the ChromaDB vector database containing the product catalog.
- **Functionality:** Answers user queries about specifications, prices, and recommends products based on semantic similarity.
- **Session Memory:** Retains context for a multi-turn conversational experience.
- **Escalation:** Detects frustration or direct requests for human help and provides support contact details.

12. NAVIGATION & RESPONSIVE DESIGN

- **Desktop Navbar:** Horizontal layout with mega-menus and search.
- **Orders/Returns Sub-nav:** A dedicated pill-shaped navigation bar placed below the main header. Disappears on downward scroll and reappears on upward scroll.
- **Mobile Navbar:** A hamburger menu triggering a slide-out drawer. The Orders/Returns links are intentionally moved out of the mobile drawer and kept in the external sub-bar for accessibility.
- **Floating Buttons:** Chatbot and Cart buttons use fixed positioning and adjust their bottom offset (`isFooterInView`) to prevent overlapping the footer.

13. DATABASE & DATA FLOW

Data Flow Diagram:
WooCommerce (MySQL) → WP REST API → React Frontend (SWR Cache) → User Interface
WooCommerce Webhook → FastAPI Backend → ChromaDB (Embeddings) → RAG Chatbot

- **Products / Categories / Reviews / Orders:** Sourced as the single source of truth from WooCommerce MySQL tables.
- **Vector Embeddings:** Stored in ChromaDB (`modena_products_v1` collection) to feed the AI context.

14. API DOCUMENTATION

Frontend to WordPress/WooCommerce APIs

- `GET /wp-json/wc/store/v1/products`: Fetch product catalog.
- `GET /wp-json/wc/v3/products/reviews`: Fetch reviews.
- `POST /wp-json/jwt-auth/v1/token`: Obtain JWT token.
- `POST /wp-json/modena/v1/check-user-exists`: Verify if email is registered.
- `POST /wp-json/modena/v1/send-otp`: Trigger email OTP.
- `POST /wp-json/modena/v1/verify-otp-register`: Validate OTP and register.
- `POST /wp-json/modena/v1/create-razorpay-order`: Generate payment intent.
- `POST /wp-json/modena/v1/upload-return-proof`: Upload return images.

Frontend to FastAPI (RAG)

- `POST /api/v1/chat/assistant`: Submit user message. Body: `{ session_id: string, message: string }`.
- `GET /api/v1/admin/rag-eval`: Retrieve analytics and logs.

WooCommerce to FastAPI (Webhook)

- `POST /api/v1/webhook/woocommerce/product-update`: Ingest product data into ChromaDB.

15. COMPONENT DOCUMENTATION

- **`App.jsx`**: The root component managing global state (cart, view, auth), rendering main layouts, and holding the Lightbox Image Viewer.
- **`HeroBanner.jsx`**: Fetches and renders the top slider from the `hero-banner` category.
- **`ProductList.jsx`**: The dynamic grid rendering products based on category filters.
- **`Chatbot.jsx`**: The floating UI handling message state and FastAPI interactions.
- **`useProducts.js`**: Custom hook encapsulating SWR logic for fetching and normalizing WooCommerce products.

16. BACKEND DOCUMENTATION (FastAPI)

- **Architecture:** Built with Python FastAPI, utilizing a modular structure (`main.py`, `config.py`, `rag_chain.py`, `ingester.py`).
- **Database:** Local SQLite-backed ChromaDB for vector storage.
- **Security:** CORS configured to accept frontend origins. Payload validation via Pydantic models.

17. WORDPRESS / WOOCOMMERCE CONFIGURATION

- **Categories:** Must have `hero-banner` and `out-of-stock` categories configured.
- **Custom Endpoints:** Provided by custom plugin/functions extending the WP REST API under the `/modena/v1/` namespace.
- **Auth:** Requires the JWT Authentication for WP REST API plugin.

18. ENVIRONMENT VARIABLES & CONFIGURATION

FastAPI (`.env`):

- `PROJECT_NAME`
- `VERSION`
- `SMTP_SERVER`
- `SMTP_PORT`
- `SMTP_USERNAME`
- `SMTP_PASSWORD`
- `EMAILS_FROM_NAME`
- `EMAILS_FROM_EMAIL`

(Note: Actual keys and passwords are [REDACTED] in documentation and source control).

19. SECURITY

- **Authentication:** Uses secure JWT tokens stored locally.
- **Payment Security:** Razorpay handles PCI compliance; the backend strictly verifies the cryptographically signed payment signature before creating orders.
- **CORS:** The FastAPI backend is configured to accept cross-origin requests from the React frontend.
- **Sanitization:** HTML strings from WordPress are aggressively sanitized and decoded in React to prevent XSS.

20. DEPLOYMENT

- **Frontend Build:** Executed via `npm run build` using Vite. Output in `/dist/`.
- **Hosting:** Frontend static files can be served via Nginx/Apache, Vercel, or integrated into the WP theme directory.

- **Backend:** FastAPI runs via Uvicorn (`python -m uvicorn main:app --host 127.0.0.1 --port 8000`). Needs to be deployed as a background service (e.g., systemd, Docker).
- **Database:** Standard WordPress MySQL/MariaDB deployment.

21. PROJECT FOLDER STRUCTURE

```
modena-react-theme/
├── src/
│   ├── components/
│   │   ├── Home/
│   │   ├── Auth/
│   │   ├── Checkout/
│   │   ├── Account/
│   │   └── Reviews/
│   ├── hooks/
│   │   └── useProducts.js
│   ├── App.jsx
│   ├── Chatbot.jsx
│   └── main.jsx
├── rag-backend/
│   ├── chroma_data/
│   ├── routers/
│   ├── main.py
│   ├── config.py
│   └── rag_chain.py
├── functions.php
├── package.json
└── vite.config.js
```

22. USER FLOW

Standard Purchase Journey:
Visitor → Views Homepage Hero Banner → Browses Signature Collection → Clicks Product → Views Details & Pinch-Zooms Image → Adds to Cart → Opens Cart Drawer → Authenticates (OTP) → Completes Razorpay Checkout → Views Order in Dashboard.

Support Journey:
Customer → Opens Chatbot → Asks question about product → AI responds based on Vector DB → Customer gets instant clarification.

23. ERROR HANDLING & TROUBLESHOOTING

- **WooCommerce API Down:** Frontend gracefully displays fallback skeleton loaders or empty states.
- **FastAPI Down:** Chatbot displays a polite "Agent offline" message.
- **Image Loading Issues:** React `onError` handlers replace broken URLs with high-quality fallback placeholder images.
- **Payment Failure:** Razorpay modal closes and the user is kept in the cart state without losing data.

24. MAINTENANCE GUIDE

- **Adding Hero Products:** Log into WordPress, create/edit a product, and assign the category `hero-banner`.
- **Marking Out of Stock:** Assign the category `out-of-stock` in WordPress.
- **Updating AI Context:** Trigger the WooCommerce webhook or restart the FastAPI `ingester.py` script to rebuild the ChromaDB index.
- **Deploying Updates:** Run `npm run build` and ensure the `dist` contents are correctly served by the web server.

25. KNOWN LIMITATIONS

- State management relies heavily on prop drilling and localized state in `App.jsx` rather than a global context provider (e.g., Redux or Zustand).
- "Not implemented": Direct user password reset flow is handled via OTP registration/login hybrid, bypassing traditional password recovery.

26. FUTURE IMPROVEMENTS

- **High Priority:** Implement a global state manager (Zustand) to clean up `App.jsx` prop drilling.
- **Medium Priority:** Add Server-Side Rendering (SSR) via Next.js for improved SEO metrics.
- **Optional:** Add multi-language support (i18n).

27. FINAL SYSTEM SUMMARY

The Modena Platform is a highly decoupled, modern e-commerce solution. By utilizing React for a fluid, physics-based responsive UI, WordPress as a robust headless CMS, and a dedicated Python FastAPI microservice for an intelligent RAG Chatbot, the system delivers an enterprise-grade, "Apple-like" premium shopping experience that is both highly performant and scalable.